

Judith Barrios

ITAI 3377

Professor Sitaram Ayyagari

01/28/26

Introduction

This lab focuses on converting and deploying an AI model using TensorFlow Lite. TensorFlow Lite is important because it allows trained AI models to run on smaller devices such as phones, sensors, and edge devices, these devices do not have a lot of memory or processing power, so the model must be lightweight and efficient. In this lab, I trained a simple neural network using the MNIST dataset, converted the model into TensorFlow Lite format, and tested it using the TensorFlow Lite interpreter. This simulates how AI models are deployed on real edge devices.

Tool Used: Google Colab

Description

Google Colab is an online coding environment that allows Python code to run without installing software.

Purpose

It is used to train and test AI models quickly and easily using cloud resources.

Use in This Lab

- Run TensorFlow code
- Train the MNIST model
- Convert the model to TensorFlow Lite
- Test the converted model

Step 1: Checking Python and TensorFlow

What was done

- Verified Python version
- Verified TensorFlow installation

Why it matters

TensorFlow must be installed to train and convert AI models.

```
!python --version !pip show tensorflow
```

```
Python 3.12.12
```

Step 2: Loading and Preparing MNIST

What is MNIST

MNIST is a dataset of handwritten digits (0-9). Each image is 28x28 pixels.

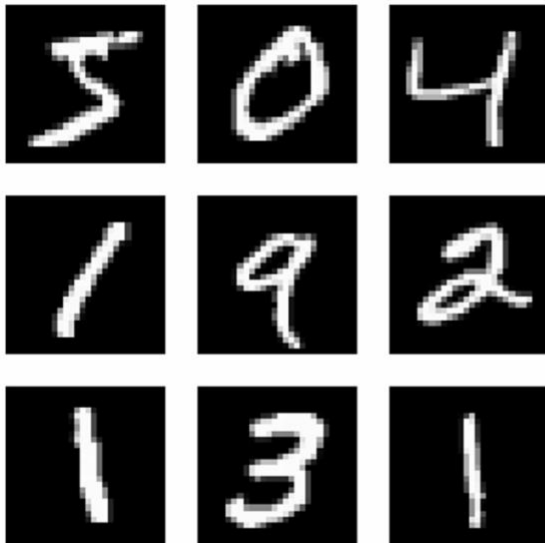
What was done

- Loaded training and testing data
- Normalized pixel values from 0-255 to 0-1
- Displayed sample images

Why it matters

Normalization helps the model learn faster and more accurately.

```
plt.show()
```



Step 3: Building the Neural Network

Model Structure

- Flatten Layer - Converts 28x28 image into a single line of numbers
- Dense Layer (128 neurons, ReLU) - Learns patterns
- Output Layer (10 neurons, Softmax) - Predicts digit 0-9

Why this model works

It is simple but effective for digit classification.

```
# Define model architecture
model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28,28)), # Input layer (flatten 28x28 pixels)
    tf.keras.layers.Dense(128, activation='relu'), # Hidden layer
    tf.keras.layers.Dense(10, activation='softmax') # Output layer (10 classes)
])
```

Step 4: Training the Model

What was done

- Optimizer: Adam
- Loss: Sparse Categorical Crossentropy
- Metric: Accuracy
- Epochs: 5

What happened

The model improved accuracy each epoch and learned how to recognize digits.

```
/usr/local/lib/python3.14/dist-packages/keras/src/layers/rescaling/Rescaling.py:27: UserWarning: Do not pass an input_shape / input_tensor argument to Rescaling layer.
super().__init__(**kwargs)
Epoch 1/5
1875/1875 — 10s 4ms/step - accuracy: 0.8710 - loss: 0.4412 - val_accuracy: 0.9595 - val_loss: 0.1310
Epoch 2/5
1875/1875 — 7s 4ms/step - accuracy: 0.9644 - loss: 0.1191 - val_accuracy: 0.9723 - val_loss: 0.0975
Epoch 3/5
1875/1875 — 8s 4ms/step - accuracy: 0.9769 - loss: 0.0768 - val_accuracy: 0.9751 - val_loss: 0.0806
Epoch 4/5
1875/1875 — 6s 3ms/step - accuracy: 0.9845 - loss: 0.0517 - val_accuracy: 0.9725 - val_loss: 0.0823
```

Step 5: Saving the model

File Saved

mnist_model.h5

Why

This allows the trained model to be reused or converted.

```
Epoch 5/5  
1875/1875  8s 4ms/step - accuracy: 0.9885 - loss: 0.0391 - val_accuracy: 0.9781 - val_loss: 0.0734  
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format  
Model training complete and saved as mnist_model.h5
```

Step 6: Converting to TensorFlow Lite

What was done

The model was converted into:

`mnist_model.tflite`

Why TensorFlow Lite is important

- Smaller file size
- Faster inference
- Designed for edge devices

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until  
Saved artifact at '/tmp/tmp0ca_derk'. The following endpoints are available:  
  
* Endpoint 'serve'  
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.float32, name='input_layer')  
Output Type:  
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)  
Captures:  
  133426961129616: TensorSpec(shape=(), dtype=tf.resource, name=None)  
  133426961128464: TensorSpec(shape=(), dtype=tf.resource, name=None)  
  133426961129232: TensorSpec(shape=(), dtype=tf.resource, name=None)  
  133426961122128: TensorSpec(shape=(), dtype=tf.resource, name=None)  
Model successfully converted and saved as mnist_model.tflite
```

Step 7: Loading the TensorFlow Lite Model

What was done

- Loaded model using “tf.lite.Interpreter”
- Allocated tensors
- Checked input and output shapes

```
Input Details: [{'name': 'serving_default_input_layer:0', 'index': 0, 'shape': array([ 1, 28, 28], dtype=int32), 'shape_signature': a
Output Details: [{'name': 'StatefulPartitionedCall_1:0', 'index': 0, 'shape': array([ 1, 10], dtype=int32), 'shape_signature': a
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning: Warning: tf.lite.Interpreter
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](https://ai.google.dev/edge/litert/migration)
for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)
```

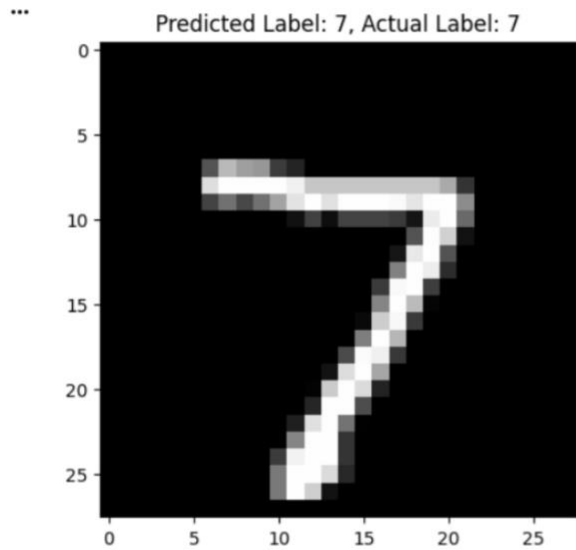
This confirms the model is ready to run on an edge device.

Step 8: Running Inference

What was done Loaded

- Selected a test image
- Sent it to the TFLite model
- Got predicted label
- Compared with actual label

```
# Display the image and prediction
plt.imshow(x_test[0], cmap="gray")
plt.title(f"Predicted Label: {predicted_label}, Actual Label: {y_test[0]}")
plt.show()
```



Result

The predicted number matches or closely matched the real value, showing the model works.

Why This Simulates Edge AI

Even though we used a computer, this is the same process used on:

- Smartphones
- Raspberry Pi
- Smart cameras

- Medical devices
- Wearables

The. tflite model is what actually runs on those devices.